

Web Components II

10 ejercicios prácticos — comunicación entre componentes y la librería Lit

DOCUMENTO DE REFERENCIA

Frontend 05

Estos ejercicios repasan la comunicación entre web components (atributos, eventos personalizados, bubbling, gestores de estado) y los fundamentos de la librería Lit.

Fácil (1-3)

Fácil-Medio (4-7)

Medio (8-10)

1 Comunicación Parent-to-Child

FÁCIL

Explica qué es la comunicación Parent-to-Child entre web components y pon un ejemplo con un componente padre `<formulario-envio>` y uno hijo `<campo-input>`.

2 La propiedad bubbles

FÁCIL

Explica qué controla la propiedad `bubbles` al crear un `CustomEvent`, y qué ocurre si se deja en su valor por defecto.

3 Crear un CustomEvent

FÁCIL

Escribe el código para crear y emitir un `CustomEvent` llamado `'producto-agregado'` que incluya en su `detail` un objeto con la propiedad `id: 42`.

4 Detener la propagación

FÁCIL-MEDIO

Explica qué hace `event.stopPropagation()` y describe un escenario en el que tendría sentido usarlo dentro de un web component.

5 Patrón observador vs PubSub

FÁCIL-MEDIO

Explica la diferencia principal entre el patrón observador y el patrón PubSub (Publisher/Subscriber) según el documento.

6 Gestor de estados

MEDIO

Explica qué es un gestor de estados y en qué tipo de aplicación (sencilla o compleja) tendría sentido introducir una herramienta especializada como Redux en lugar de usar solo el contexto global.

7 Instalar Lit y crear una clase base

MEDIO

Escribe el comando npm para instalar Lit y el código base de una clase `MiComponente` que extienda de `LitElement` e importe correctamente la clase.

8 Propiedades reactivas en Lit

MEDIO

Define, usando `static properties`, una propiedad reactiva llamada `contador` de tipo `Number` asociada a un atributo, e inicialízala a 0 en el constructor.

9 Ciclo de actualización en Lit

MEDIO

Explica, en orden, para qué sirven los métodos `shouldUpdate`, `willUpdate`, `update` y `render` dentro del ciclo de actualización de un componente Lit.

10 Ejercicio integrador

MEDIO

Diseña la comunicación entre un componente padre `<lista-tareas>` y un componente hijo `<tarea-item>`: el padre debe poder marcar una tarea como prioritaria modificando un atributo del hijo, y el hijo debe poder avisar al padre, mediante un evento personalizado con bubbling y composed activados, cuando el usuario la marca como completada. Describe el flujo completo.

Soluciones orientativas

1. Es la comunicación en la que el componente padre modifica un atributo del componente hijo para que este reaccione; por ejemplo, `formulario-envio` podría cambiar el atributo `valor` de `campo-input` para autorellenarlo.
2. `bubbles` indica si el evento se propagará hacia arriba por el árbol del DOM (event bubbling). Por defecto es `false`, por lo que el evento no se propagaría a los elementos padres salvo que se active explícitamente.
3.

```
const evento = new CustomEvent('producto-agregado', { detail: { id: 42 } }); this.dispatchEvent(evento);
```
4. Detiene la propagación del evento hacia los elementos padre. Tendría sentido, por ejemplo, si un componente hijo no quiere que un clic interno también dispare la lógica de un listener del componente padre.
5. En el patrón observador, el Subject conoce y notifica directamente a sus observadores; en PubSub existe un canal/intermediario que gestiona las suscripciones, de forma que el publicador y los suscriptores no se conocen entre sí.
6. Un gestor de estados administra el conjunto de valores de la aplicación, permitiendo modificarlos, consultarlos y reaccionar a sus cambios. En aplicaciones sencillas basta con el contexto global y algún patrón de notificación; en aplicaciones más complejas conviene una herramienta como Redux, que registra todos los cambios (síncronos y asíncronos) de forma trazable.
7. `npm i lit`. Código base:

```
import { LitElement } from 'lit'; class MiComponente extends LitElement { } customElements.define('mi-componente', MiComponente);
```
8.

```
static properties = { contador: { type: Number } }; constructor(){ super(); this.contador = 0; }
```
9. `shouldUpdate` decide si el componente debe actualizarse; `willUpdate` se ejecuta cuando ya se sabe que habrá actualización y sirve para calcular valores previos; `update` refleja las propiedades en los atributos y llama a `render`; `render` genera la plantilla HTML final del componente.
10. El padre `lista-tareas` establece el atributo `prioritaria` en el hijo correspondiente (comunicación Parent-to-Child); el hijo `tarea-item`, al detectar el clic de "completada", crea un `CustomEvent('tarea-completada', { bubbles:true, composed:true, detail:{id: this.id}})` y lo emite con `dispatchEvent`; el padre escucha ese evento con `addEventListener` y actualiza su propio estado en consecuencia.

Practica primero sin mirar las soluciones — es la mejor forma de consolidar lo aprendido.